

Sesión 5

Estado de la aplicación

Fase 2 — Construir el sistema base

Duración: 1 hora

Recap rápido — Sesión 4

Antes de empezar, recordamos en voz alta lo que construimos la semana pasada:

Patrón fundamental que aprendiste:

evento → función → estado cambia → UI se actualiza

Lo que construiste:

- Un objeto de estado en JavaScript
- Un botón con un evento click
- Una función que cambiaba el estado
- Una UI que reflejaba ese cambio

Archivos que tienes:

- index.html → estructura
- estado.js → dónde vive la información
- app.js → comportamiento

Pregunta de calentamiento (1 minuto):

¿Qué pasaría si alguien quitara el archivo estado.js? ¿Seguiría funcionando la app?

1. Idea central de la sesión

"Una app no es solo botones y colores. Es información que cambia con el tiempo. El estado es esa información."

Al final de esta hora debes poder responder con seguridad:

- ¿Qué es el estado de una aplicación?
- ¿Dónde debe vivir ese estado?

- ¿Qué pasa cuando el estado se descontrola?
-

2. Conexión con lo que ya sabes

En Scratch

En Scratch usabas variables para guardar información: la puntuación, el número de vidas, si el juego había terminado. Esas variables eran el estado de tu juego. El problema es que en Scratch era difícil saber qué variable cambiaba qué cosa. Todo estaba mezclado.

En HTML y CSS

HTML y CSS no tienen estado. Una página HTML siempre muestra lo mismo. Si quieres que algo cambie, necesitas JavaScript. Por eso aprendimos JS en la sesión 4.

En ingeniería real

Todas las apps del mundo tienen estado: Instagram sabe si has dado like o no. WhatsApp sabe qué mensajes has leído. Tu app de mini-juegos necesita saber en qué pantalla estás, qué juego está activo, qué puntuación tienes. Sin estado, la app no tiene memoria de nada.

3. Qué es el estado

El estado es la información que describe cómo está tu app en un momento concreto.

Imagina que congelas la app en este segundo. Si pudieras mirar dentro y ver todos los datos, eso sería el estado.

Ejemplo con tu app

El estado de tu app podría ser así:

```
estado = { pantallaActual: "menu", juegoActivo: null, puntuacion: 0,
turno: null }
```

Este objeto responde preguntas como:

- ¿En qué pantalla estamos? → pantallaActual
- ¿Hay un juego en marcha? → juegoActivo
- ¿Cuántos puntos lleva el jugador? → puntuacion
- ¿De quién es el turno ahora? → turno

Estado vs. apariencia

Estado → información (en JavaScript)

Apariencia → cómo se ve ese estado (en HTML/CSS)

La UI no inventa información. Solo refleja lo que dice el estado.

4. Diseño antes de código

Antes de escribir una sola línea, vamos a pensar qué partes del estado necesita nuestra app.

Qué partes tiene el estado de tu app

Propiedad	Qué representa	Quién la cambia
pantallaActual	En qué pantalla está el usuario	El sistema de navegación
juegoActivo	Qué juego está en marcha (o ninguno)	El menú cuando el usuario elige
puntuacion	Puntos del jugador en la sesión actual	La lógica de cada juego
turno	De quién es el turno en el juego actual	La lógica del juego activo

Regla de oro del estado

El estado vive en un solo lugar. Nadie lo cambia directamente, solo las funciones autorizadas.

Si el estado vive disperso en 10 archivos distintos, cuando algo falle no sabrás dónde buscar.

5. Ejemplo aplicado al proyecto

Vamos a ver cómo funciona el estado en un momento concreto de tu app: cuando el usuario pulsa "Jugar al Tic Tac Toe" desde el menú.

Antes de pulsar el botón

```
estado = { pantallaActual: "menu", juegoActivo: null, puntuacion: 0 }
```

Después de pulsar el botón

```
estado = { pantallaActual: "juego", juegoActivo: "tictactoe",  
puntuacion: 0 }
```

La UI detecta ese cambio y muestra la pantalla del juego en vez del menú. La información cambió, la pantalla la refleja.

El flujo completo

1. Usuario pulsa el botón "Tic Tac Toe"
2. Se dispara un evento click
3. Una función llamada iniciarJuego() se ejecuta
4. Esa función modifica el estado: juegoActivo = 'tictactoe'
5. La UI detecta el cambio y renderiza la pantalla correcta

6. Uso guiado de agentes de IA

Esta semana vamos a usar la IA para una tarea concreta. Pero primero decides tú, luego la IA ayuda.

Qué pedirle

Prompt recomendado:

```
Tengo una app de mini-juegos con HTML, CSS y JavaScript. Necesito un  
archivo estado.js que guarde el estado global. El estado debe tener:  
pantallaActual, juegoActivo y puntuacion. Incluye una función para  
cambiar de pantalla y otra para actualizar la puntuación. No mezcles  
lógica de juego en este archivo.
```

Qué revisar obligatoriamente

- ¿El estado es un solo objeto centralizado o hay variables sueltas por todos lados?
- ¿Las funciones modifican el estado o lo reemplazan entero?
- ¿Hay lógica de juego mezclada donde no debería estar?
- ¿Los nombres tienen sentido? ¿Se entienden sin explicación?

Errores típicos de la IA en este contexto

- **Variables globales sueltas:** en vez de un objeto estado, la IA pone let pantalla = ..., let juego = ... por separado.
- **Mezcla responsabilidades:** mete reglas de Tic Tac Toe dentro del archivo de estado.
- **Sobrescribe el estado completo:** en vez de cambiar solo lo necesario, reemplaza el objeto entero.
- **Nombres crípticos:** usa nombres como s, g, p en vez de pantallaActual, juegoActivo, puntuacion.

7. Preguntas de pensamiento crítico

No son de memoria. Son para pensar. No hay respuesta única.

6. Si dos juegos distintos pueden cambiar la puntuación, ¿dónde debería vivir la lógica de puntuación?
7. ¿Qué pasaría si cada juego tuviera su propio estado por separado, sin un estado global? ¿Cuándo sería un problema?
8. Tienes una propiedad en el estado que se llama "pantalla". ¿Qué nombre sería más claro: pantalla, vistaActual, pantallaActual, screen? ¿Por qué?
9. Si un bug hace que juegoActivo nunca vuelva a null cuando termina una partida, ¿qué problemas podría causar?
10. ¿Por qué es importante que solo funciones específicas puedan cambiar el estado y no cualquier parte del código?

8. Mini-reto de la sesión

Objetivo:

Construir un estado global sencillo para tu app y demostrar que funciona con dos pantallas.

Pasos (pero decide antes de ejecutar)

11. Antes de tocar código: dibuja en papel qué propiedades necesita tu estado y por qué cada una.
12. Crea estado.js con un objeto de estado y dos funciones: irAMenu() e iniciarJuego(nombre).
13. Crea index.html con dos secciones: una para el menú y otra para el juego (puedes ocultarlas con CSS).
14. Conecta todo en app.js: un botón que llame a iniciarJuego() y cambie lo que ve el usuario.
15. Prueba el flujo completo: menú → juego → menú.

Criterio de éxito

No es que funcione.

Es que puedas explicar qué hace cada archivo y por qué el estado vive donde vive.

Pregunta final de verificación:

- ¿Qué archivo tiene el estado? ¿Por qué está ahí y no en otro sitio?
- Si mañana quisieras añadir un tercer juego, ¿qué tendrías que cambiar en el estado?

9. Qué NO hacer

Errores de principiantes

- Poner el estado dentro de index.html como variables sueltas en un `<script>`.
- Cambiar el estado directamente desde la UI: el botón no debería saber nada del estado, solo llamar a una función.
- Tener un archivo gigante con todo mezclado: HTML, CSS, estado y lógica juntos.
- Copiar código de la IA sin leerlo. Si no lo entiendes, no lo uses todavía.

Errores de la IA que debes evitar aceptar

- Código que "funciona pero es un caos por dentro". Funcionar no es suficiente.
- Estado distribuido en muchas variables sin estructura. La IA a veces hace esto.
- Funciones con nombres genéricos como `handleClick`, `doThing`, `update`. Pide nombres descriptivos.
- Lógica de un juego específico metida en el sistema de estado global.

Lo que debe quedar claro al salir

- El estado es la información que describe tu app en un momento dado.
- Vive en un solo lugar: un objeto central en un archivo dedicado.
- Solo funciones concretas pueden cambiarlo.
- La UI refleja el estado; no lo decide.
- Si el estado se descontrola, la app falla de formas difíciles de encontrar.

Próxima sesión: Navegación entre pantallas